Backend de UNAB GO!: API REST para la Vida Universitaria de la Universidad Autónoma de Bucaramanga

Primera Entrega: Propuesta y Arquitectura

Juan Sebastián Lizcano Jaimes

Miguel Angel Hernandez Quintero

Juan Alejandro Sierra Rincon

Facultad de Ingeniería, Universidad Autónoma de Bucaramanga

Desarrollo y Arquitectura Backend

MSc. Feisar Moreno

3 de noviembre de 2026

Backend de UNAB GO!: API REST para la Vida Universitaria de la Universidad Autónoma de Bucaramanga

UNAB GO! es una aplicación móvil para Android, desarrollada en Kotlin, que reúne en un solo lugar los servicios académicos, institucionales y de bienestar que un estudiante de la Universidad Autónoma de Bucaramanga (UNAB) usa cada semana: horario con recordatorios, calculadora de notas según el Reglamento Estudiantil, mapa en vivo de los buses de las Rutas UNAB, avisos, oferta académica y un asistente con inteligencia artificial llamado Banu. La aplicación ya existe y funciona; hoy guarda los datos académicos solo en el teléfono y cuenta con un servidor ligero que protege las claves de la inteligencia artificial y entrega la posición de los buses.

Este documento presenta la propuesta y la arquitectura del backend que se construirá como proyecto final de la asignatura. El backend convierte a UNAB GO! en un sistema multiusuario con cuentas, tres roles con permisos distintos, una base de datos relacional y una API REST que la aplicación móvil consumirá como su capa de presentación. El documento describe el dominio de negocio, los actores y roles, el esquema de datos, las APIs externas que se integrarán y los endpoints REST previstos.

Descripción del Dominio de Negocio

Planteamiento del Problema

La información y los servicios que un estudiante de la UNAB necesita están repartidos en más de nueve canales: el sitio web institucional, miportalU, la plataforma TEMA, Cosmos, la biblioteca, el carné digital, Bienestar Universitario, UNAB Radio y el Reglamento Estudiantil en PDF. El estudiante no recibe avisos antes de sus clases, calcula a mano la nota que necesita para aprobar con reglas que pocos conocen (por ejemplo, que la nota final se registra en décimas sin

aproximar), no sabe dónde va el bus entre sedes y se entera tarde de los avisos de las dependencias porque se pierden en las redes sociales.

Las dependencias tienen el problema inverso: Bienestar, Comunicaciones y Transporte no cuentan con un canal propio para llegar al teléfono del estudiante con un aviso oportuno, y la información que publican (preguntas frecuentes, rutas, horarios) queda dispersa y desactualizada. En una encuesta a 50 estudiantes, después de probar la aplicación, 48 (96 %) respondieron que la usarían y los 50 (100 %) la consideraron necesaria en la UNAB.

Justificación del Sistema

La versión actual de UNAB GO! resuelve la experiencia del estudiante en un solo teléfono, pero no puede crecer sin un backend. Se necesita un sistema del lado del servidor por cinco razones:

- **Cuentas y sincronización:** el horario, las materias y las notas se pierden si el estudiante cambia de teléfono; con el backend quedan respaldados y disponibles en cualquier dispositivo.
- **Gestión de contenidos por roles:** las dependencias deben publicar avisos y mantener las preguntas frecuentes de Banu sin que haya que publicar una nueva versión de la aplicación.
- **Notificaciones:** al publicar un aviso, el backend envía la notificación push a los estudiantes.
- **Integración centralizada:** el GPS de los buses y la inteligencia artificial se consultan una sola vez desde el servidor y se reparten entre todos los usuarios, con caché y sin exponer credenciales en el teléfono.

- **Seguridad y privacidad:** el acceso a cada recurso se controla por rol y por propiedad, conforme a la Ley 1581 de 2012 de protección de datos personales (Congreso de la República de Colombia, 2012).

Reglas de Negocio

El sistema no es un CRUD trivial porque cada operación aplica reglas propias del dominio universitario:

- **Notas según el Reglamento Estudiantil** (Universidad Autónoma de Bucaramanga, 2019): la nota final se registra en unidades y décimas sin aproximar (Art. 39), cada corte debe tener al menos dos notas (Art. 44), los pesos de los cortes de una materia no pueden superar el 100 % y el sistema calcula el promedio, lo acumulado y la nota que falta para aprobar con 3,0.
- **Horario sin choques:** un estudiante no puede registrar dos bloques de clase que se crucen el mismo día.
- **Estado de los buses:** a partir de dos reportes del GPS se calcula si cada bus está en movimiento (velocidad y rumbo), detenido, en servicio o estacionado, y cuál es la sede más cercana; los reportes de más de 5 minutos (30 si el bus está apagado) se descartan.
- **Avisos con ciclo de vida:** un aviso nace como borrador, se publica y al publicarse dispara una sola notificación push.
- **Respuestas verificadas de Banu:** las preguntas frecuentes vigentes responden sin inteligencia artificial; cuando se usa la IA, toda cifra de la respuesta debe existir en la fuente oficial y, si un modelo falla, se pasa al siguiente.

- **Autorización por propiedad:** un estudiante solo ve y modifica sus propias materias; un gestor solo edita sus avisos; el administrador gestiona usuarios, pero no lee las notas de los estudiantes.

Arquitectura y Tecnología Propuesta

Se usará **Kotlin con el framework Ktor** (JetBrains, s. f.-a), porque el servidor de UNAB GO! ya está escrito en Kotlin y comparte con la aplicación Android un módulo común (Kotlin Multiplatform) con los modelos de datos, la lógica de notas y el lector del GPS. Así el backend y la aplicación móvil no pueden quedar desalineados. Cada requisito técnico del curso tiene un equivalente directo, como muestra la Tabla 1.

La arquitectura es en capas: las **rutas y controladores** reciben la petición y la validan; los **servicios de dominio** aplican las reglas de negocio; los **repositorios** acceden a la base de datos mediante el ORM; y los **servicios de integración** encapsulan las llamadas a las APIs externas. Ningún controlador ni modelo hace llamadas HTTP directas.

Tabla 1

Equivalencia entre los Requisitos del Curso en Laravel y su Implementación en Kotlin con Ktor

Requisito del curso (Laravel)	**Implementación en el backend de UNAB GO!**
PHP y Laravel	Kotlin 2.2 y Ktor 3.3 sobre la JVM 21
ORM Eloquent con $fillable y relaciones	Exposed, el ORM de JetBrains (s. f.-b): entidades con relaciones uno a uno, uno a muchos y muchos a muchos; la asignación masiva se controla con objetos de transferencia (DTO)
Migraciones	Flyway (Redgate, s. f.): archivos SQL versionados (V1__crear_roles.sql…)
Factories y DatabaseSeeder	Fábricas con datos de prueba (Datafaker) y DatabaseSeeder.kt; la tarea dbFreshSeed equivale a migrate:fresh --seed
Laravel Sanctum	Tokens personales propios: se entrega un token aleatorio y en la tabla access_tokens solo se guarda su huella SHA-256; plugin Authentication (bearer) de Ktor
Gates y Policies	Clases Policy (SubjectPolicy, AnnouncementPolicy, FaqPolicy) y verificación de rol por grupo de rutas
Controladores API	Un controlador por recurso (AuthController, SubjectController, AnnouncementController…)
FormRequest	Clases de solicitud validadas con el plugin RequestValidation de Ktor y mensajes de error en español
Service Layer	Paquete services: GpsFeedService, AiGatewayService, PushNotificationService, UnabWebService
.env y config/services.php	Variables de entorno documentadas en .env.example y leídas por ServerConfig.kt
PHPUnit con SQLite en memoria	JUnit con testApplication de Ktor y base de datos H2 en memoria; Flyway migra en cada prueba
Http::fake()	MockEngine del cliente de Ktor para simular las APIs externas sin internet
Exención CSRF en webhooks	No aplica: la API no usa cookies de sesión; el webhook se valida con firma HMAC
Frontend	Aplicación Android nativa en Kotlin con Jetpack Compose, ya construida

Nota. Los motores de base de datos previstos son PostgreSQL en producción y H2 en memoria para las pruebas.

Especificación de Actores y Roles

El sistema maneja tres roles. Cada usuario tiene exactamente un rol (relación de la tabla users con la tabla roles) y cada petición autenticada lleva un token que identifica al usuario. Además, algunos recursos públicos (programas, buses en vivo, avisos publicados y la consulta a Banu con límites estrictos) se pueden consultar sin iniciar sesión, por ejemplo por un aspirante; ese acceso no es un rol, sino la ausencia de sesión.

Tabla 2

Roles del Sistema y Responsabilidades

Rol	Quién es	Qué puede hacer
Estudiante (usuario final)	Estudiante con correo institucional @unab.edu.co. Se registra por sí mismo o entra con su cuenta de Google institucional.	Gestionar su perfil, sus materias, su horario y sus notas; consultar el resumen de notas; guardar rutas de bus favoritas; ver los buses en vivo; leer avisos; preguntar a Banu.
Gestor (dependencia)	Funcionario de Bienestar, Comunicaciones o Transporte. El administrador le asigna el rol.	Crear, publicar, editar y retirar sus avisos (con notificación push); administrar las preguntas frecuentes de Banu; mantener las rutas y paradas de los buses.
Administrador	Responsable técnico del sistema.	Listar usuarios, cambiar roles, activar o desactivar cuentas; sincronizar el catálogo de programas desde unab.edu.co; editar o retirar cualquier aviso o pregunta frecuente.

Nota. El administrador no puede leer las materias ni las notas de los estudiantes (principio de mínimo privilegio).

Tabla 3

Matriz de Permisos por Recurso

Recurso y acción	Sin sesión	Estudiante	Gestor	Administrador
Consultar programas académicos	Sí	Sí	Sí	Sí
Sincronizar programas desde unab.edu.co	No	No	No	Sí
Ver buses en vivo y rutas	Sí	Sí	Sí	Sí
Editar rutas y paradas	No	No	Sí	Sí
Guardar rutas favoritas	No	Propias	Propias	Propias
Leer avisos publicados	Sí	Sí	Sí	Sí
Crear y publicar avisos	No	No	Sí	Sí
Editar o retirar avisos	No	No	Propios	Todos
Preguntar a Banu	Con límite	Sí	Sí	Sí
Administrar preguntas frecuentes	No	No	Sí	Sí
Materias, horario y notas	No	Propias	No	No
Ver y editar el perfil	No	Propio	Propio	Propio
Usuarios y roles	No	No	No	Sí

Nota. «Propias» significa que la política de autorización compara el dueño del recurso con el usuario del token; si no coinciden, la API responde 403 Forbidden.

La autorización se aplica en dos niveles. Primero, cada grupo de rutas exige un rol (por ejemplo, las rutas /api/v1/admin solo admiten administradores). Segundo, las políticas revisan la

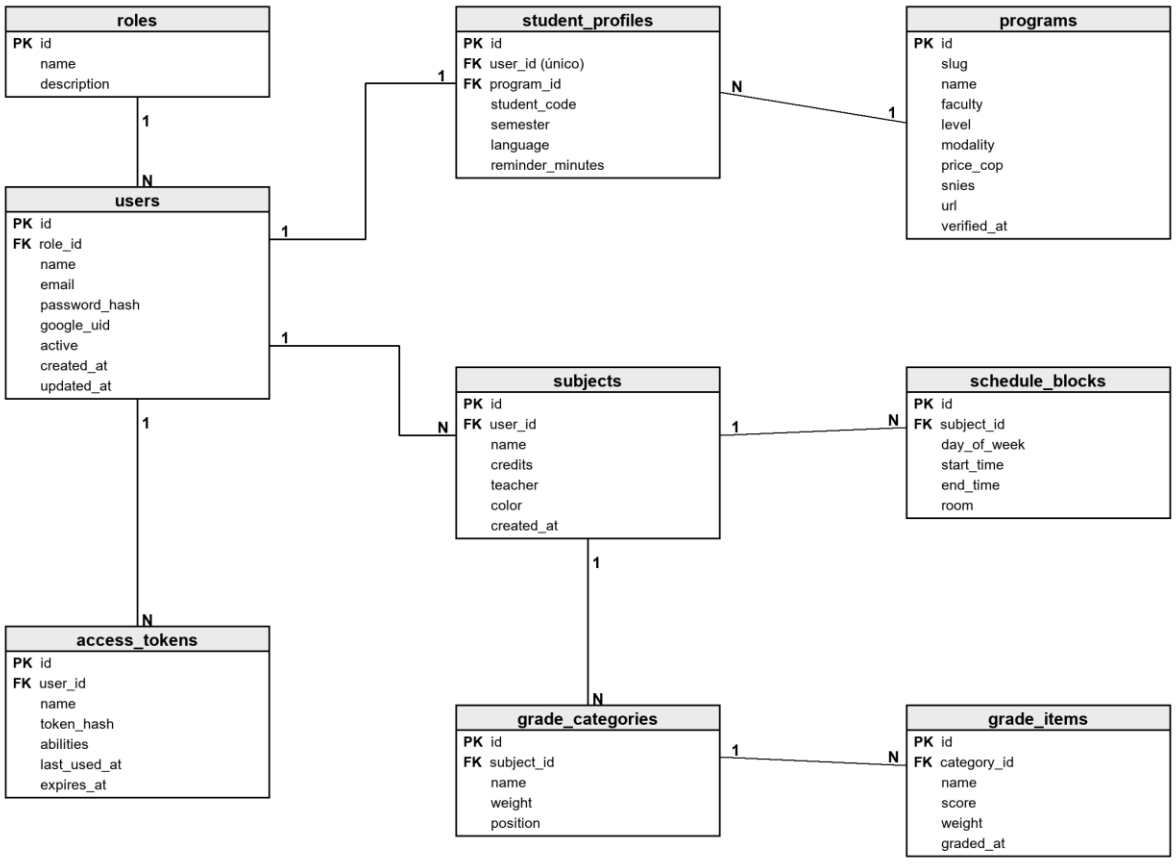
propiedad del recurso: SubjectPolicy permite ver o modificar una materia solo a su dueño, y AnnouncementPolicy permite editar un aviso a su autor o a un administrador.

Diseño del Esquema de Datos

El esquema tiene 13 modelos y 2 tablas intermedias en PostgreSQL. Incluye relaciones uno a uno (usuario y perfil de estudiante), uno a muchos (por ejemplo, materia y bloques del horario) y muchos a muchos (rutas y paradas, y usuarios y rutas favoritas). Las Figuras 1 y 2 muestran el diagrama relacional y la Tabla 4 resume las relaciones.

Figura 1

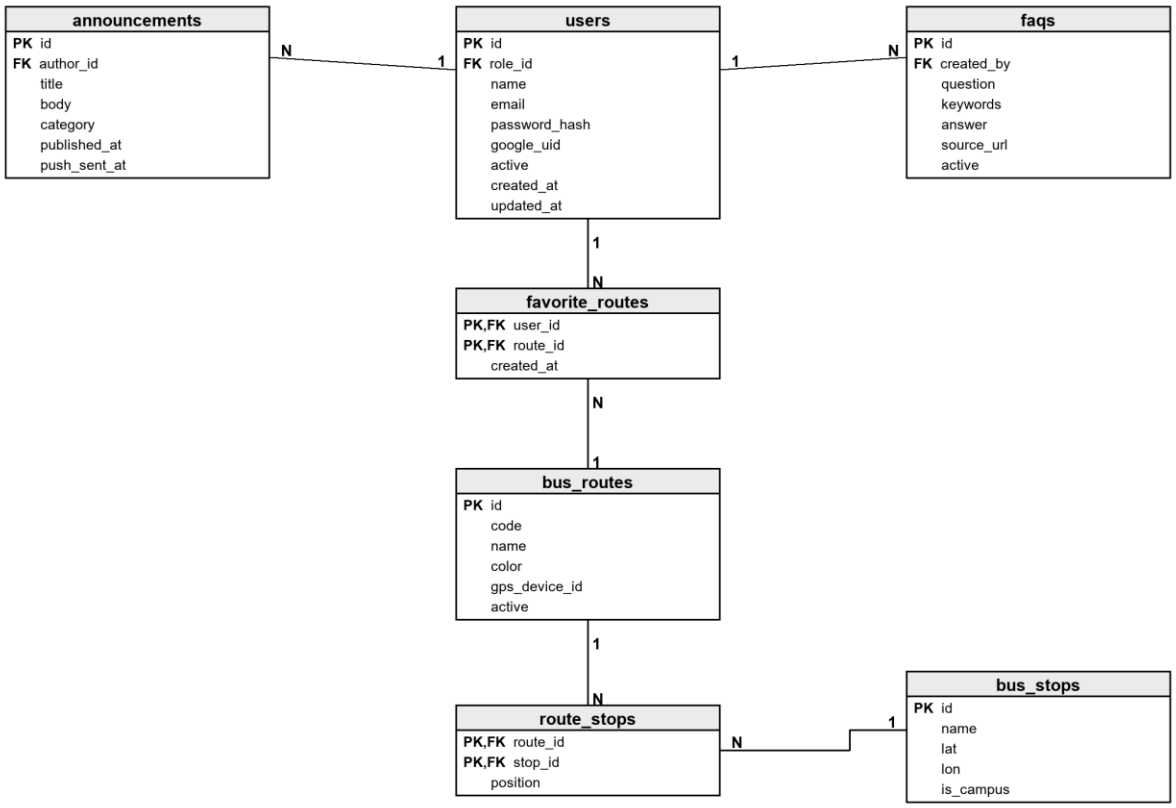
Diagrama Relacional de Usuarios, Perfiles y Gestión Académica



Nota. PK = llave primaria; FK = llave foránea; 1 y N indican la cardinalidad de cada relación.

Figura 2

Diagrama Relacional de Avisos, Preguntas Frecuentes y Buses



Nota. favorite_routes y route_stops son tablas intermedias de relaciones muchos a muchos con llave primaria compuesta.

Tabla 4

Relaciones entre los Modelos

Relación	Tipo	Descripción
roles → users	1:N	Un rol agrupa a muchos usuarios; cada usuario tiene un rol.
users → student_profiles	1:1	Cada estudiante tiene un único perfil académico.
programs → student_profiles	1:N	Un programa tiene muchos estudiantes.
users → access_tokens	1:N	Un usuario puede tener sesión en varios dispositivos.
users → subjects	1:N	Un estudiante registra muchas materias.
subjects → schedule_blocks	1:N	Una materia tiene varios bloques en el horario semanal.
subjects → grade_categories	1:N	Una materia se divide en cortes con su peso.
grade_categories → grade_items	1:N	Un corte tiene varias notas.
users → announcements	1:N	Un gestor publica muchos avisos.
users → faqs	1:N	Un gestor crea muchas preguntas frecuentes.
users ↔ bus_routes	N:M	Rutas favoritas, mediante favorite_routes.
bus_routes ↔ bus_stops	N:M	Paradas de cada ruta y su orden, mediante route_stops.

Diccionario de Datos

Las Tablas 5 a 19 definen cada tabla con el nombre del campo, su tipo en PostgreSQL, sus restricciones y su descripción. Todas las llaves foráneas de datos propios del estudiante usan borrado en cascada.

Tabla 5

Diccionario de Datos de la Tabla roles

Campo	Tipo	Restricciones	Descripción
id	SMALLSERIAL	PK	Identificador del rol.
name	VARCHAR(30)	NOT NULL, UNIQUE	ESTUDIANTE, GESTOR o ADMINISTRADOR.
description	VARCHAR(150)	NULL	Descripción del rol.

Nota. Roles del sistema.

Tabla 6

Diccionario de Datos de la Tabla users

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del usuario.
role_id	SMALLINT	FK → roles.id, NOT NULL	Rol del usuario.
name	VARCHAR(120)	NOT NULL	Nombre completo.
email	VARCHAR(160)	NOT NULL, UNIQUE	Correo institucional (@unab.edu.co).
password_hash	VARCHAR(100)	NULL	Contraseña cifrada con BCrypt; nula si entra con Google.
google_uid	VARCHAR(128)	NULL, UNIQUE	Identificador de la cuenta de Google institucional.
active	BOOLEAN	NOT NULL, DEFAULT true	Cuenta habilitada.
created_at	TIMESTAMPTZ	NOT NULL	Fecha de creación.
updated_at	TIMESTAMPTZ	NOT NULL	Fecha de la última modificación.

Nota. Cuentas de usuario.

Tabla 7

Diccionario de Datos de la Tabla student_profiles

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del perfil.
user_id	BIGINT	FK → users.id, UNIQUE, CASCADE	Dueño del perfil (relación 1:1).
program_id	BIGINT	FK → programs.id, NULL	Programa que cursa.
student_code	VARCHAR(20)	NULL, UNIQUE	Código estudiantil.
semester	SMALLINT	NULL, CHECK 1 a 12	Semestre actual.
language	VARCHAR(5)	NOT NULL, DEFAULT 'es'	Idioma: es, en, fr, pt o ko.
reminder_minutes	SMALLINT	NOT NULL, CHECK 0, 30 o 60	Minutos de anticipación del recordatorio de clase.

Nota. Perfil académico del estudiante.

Tabla 8

Diccionario de Datos de la Tabla programs

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del programa.
slug	VARCHAR(160)	NOT NULL, UNIQUE	Identificador en unab.edu.co.
name	VARCHAR(200)	NOT NULL	Nombre del programa.
faculty	VARCHAR(150)	NOT NULL	Facultad.
level	VARCHAR(40)	NOT NULL	Pregrado, posgrado, técnico o educación continua.
modality	VARCHAR(40)	NULL	Presencial, virtual o híbrida.
price_cop	BIGINT	NULL, CHECK ≥ 0	Valor del semestre en pesos.
snies	VARCHAR(20)	NULL	Código SNIES.
url	VARCHAR(300)	NOT NULL	Ficha oficial del programa.
verified_at	TIMESTAMPTZ	NULL	Última verificación del precio.

Nota. Oferta académica de la UNAB.

Tabla 9

Diccionario de Datos de la Tabla access_tokens

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del token.
user_id	BIGINT	FK → users.id, CASCADE	Dueño del token.
name	VARCHAR(60)	NOT NULL	Dispositivo (por ejemplo, «Android de Juan»).
token_hash	CHAR(64)	NOT NULL, UNIQUE	Huella SHA-256 del token; el token en claro no se guarda.
abilities	VARCHAR(200)	NOT NULL	Permisos del token.
last_used_at	TIMESTAMPTZ	NULL	Último uso.
expires_at	TIMESTAMPTZ	NOT NULL	Vencimiento.

Nota. Tokens de sesión por dispositivo.

Tabla 10

Diccionario de Datos de la Tabla subjects

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador de la materia.
user_id	BIGINT	FK → users.id, CASCADE	Estudiante dueño.
name	VARCHAR(120)	NOT NULL	Nombre de la materia.
credits	SMALLINT	NOT NULL, CHECK 0 a 10	Créditos.
teacher	VARCHAR(120)	NULL	Profesor.
color	CHAR(7)	NOT NULL	Color en el horario (#RRGGBB).
created_at	TIMESTAMPTZ	NOT NULL	Fecha de creación.

Nota. Materias que registra el estudiante.

Tabla 11

Diccionario de Datos de la Tabla schedule_blocks

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del bloque.
subject_id	BIGINT	FK → subjects.id, CASCADE	Materia.
day_of_week	SMALLINT	NOT NULL, CHECK 1 a 7	Día (1 = lunes).
start_time	TIME	NOT NULL	Hora de inicio.
end_time	TIME	NOT NULL, CHECK > start_time	Hora de fin.
room	VARCHAR(40)	NULL	Salón.

Nota. Bloques semanales de clase.

Tabla 12

Diccionario de Datos de la Tabla grade_categories

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del corte.
subject_id	BIGINT	FK → subjects.id, CASCADE	Materia.
name	VARCHAR(60)	NOT NULL	Nombre (por ejemplo, «Corte 1»).
weight	NUMERIC(5,2)	NOT NULL, CHECK 0 a 100	Peso en porcentaje; la suma por materia no supera 100.
position	SMALLINT	NOT NULL	Orden del corte.

Nota. Cortes de una materia.

Tabla 13

Diccionario de Datos de la Tabla grade_items

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador de la nota.
category_id	BIGINT	FK → grade_categories.id, CASCADE	Corte.
name	VARCHAR(80)	NOT NULL	Parcial, quiz, taller o trabajo.
score	NUMERIC(2,1)	NULL, CHECK 0,0 a 5,0	Nota; nula si está pendiente.
weight	NUMERIC(5,2)	NULL	Peso dentro del corte; nulo si todas pesan igual.
graded_at	DATE	NULL	Fecha de la evaluación.

Nota. Notas dentro de un corte.

Tabla 14

Diccionario de Datos de la Tabla bus_routes

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador de la ruta.
code	VARCHAR(20)	NOT NULL, UNIQUE	Código del bus en la plataforma GPS (por ejemplo, RUTA1).
name	VARCHAR(80)	NOT NULL	Nombre para mostrar.
color	CHAR(7)	NOT NULL	Color en el mapa.
gps_device_id	VARCHAR(40)	NULL, UNIQUE	Identificador del equipo GPS.
active	BOOLEAN	NOT NULL, DEFAULT true	Ruta en servicio.

Nota. Rutas de bus de la UNAB.

Tabla 15

Diccionario de Datos de la Tabla bus_stops

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador de la parada.
name	VARCHAR(120)	NOT NULL	Nombre (por ejemplo, «Campus El Jardín»).
lat	NUMERIC(9,6)	NOT NULL, CHECK −90 a 90	Latitud.
lon	NUMERIC(9,6)	NOT NULL, CHECK −180 a 180	Longitud.
is_campus	BOOLEAN	NOT NULL, DEFAULT false	Es una sede de la UNAB.

Nota. Paradas y sedes.

Tabla 16

Diccionario de Datos de la Tabla route_stops

Campo	Tipo	Restricciones	Descripción
route_id	BIGINT	PK, FK → bus_routes.id	Ruta.
stop_id	BIGINT	PK, FK → bus_stops.id	Parada.
position	SMALLINT	NOT NULL	Orden de la parada en la ruta.

Nota. Tabla intermedia rutas y paradas (N:M).

Tabla 17

Diccionario de Datos de la Tabla favorite_routes

Campo	Tipo	Restricciones	Descripción
user_id	BIGINT	PK, FK → users.id, CASCADE	Usuario.
route_id	BIGINT	PK, FK → bus_routes.id, CASCADE	Ruta favorita.
created_at	TIMESTAMPTZ	NOT NULL	Fecha en que se marcó.

Nota. Tabla intermedia usuarios y rutas favoritas (N:M).

Tabla 18

Diccionario de Datos de la Tabla announcements

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del aviso.
author_id	BIGINT	FK → users.id, NOT NULL	Gestor o administrador que lo creó.
title	VARCHAR(140)	NOT NULL	Título.
body	TEXT	NOT NULL	Contenido.
category	VARCHAR(30)	NOT NULL	Académico, bienestar, transporte o evento.
published_at	TIMESTAMPTZ	NULL	Fecha de publicación; nula si es borrador.
push_sent_at	TIMESTAMPTZ	NULL	Cuándo se envió la notificación push.

Nota. Avisos de las dependencias.

Tabla 19

Diccionario de Datos de la Tabla faqs

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador de la pregunta.
created_by	BIGINT	FK → users.id, NOT NULL	Gestor que la creó.
question	VARCHAR(200)	NOT NULL	Pregunta principal.
keywords	TEXT	NOT NULL	Otras formas de hacer la pregunta, separadas por punto y coma.
answer	TEXT	NOT NULL	Respuesta oficial.
source_url	VARCHAR(300)	NOT NULL	Fuente oficial de la respuesta.
active	BOOLEAN	NOT NULL, DEFAULT true	Pregunta vigente.

Nota. Preguntas frecuentes de Banu.

Plan de Integración de APIs Externas

El backend consumirá cinco servicios externos; el curso exige al menos dos. Cada integración vive en una clase de servicio propia, con tiempo máximo de espera, manejo de errores y credenciales leídas de variables de entorno. La Tabla 20 resume cada una.

Tabla 20

APIs Externas que Integra el Backend

API	Para qué se usa	Autenticación (variable de entorno)	Servicio y comportamiento
Plataforma GPS de las Rutas UNAB (gpsmobile.net)	Último reporte de cada bus: posición, rumbo, hora, encendido y evento; alimenta GET /api/v1/bus/live.	Código del cliente en la URL del servicio (GPS_FEED_URL).	GpsFeedService: consulta como máximo cada 5 s, guarda la respuesta en caché y, si el GPS falla, entrega la última posición válida durante 1 minuto.
Proveedores de IA compatibles con OpenAI (Ollama Cloud; respaldo Groq, Gemini y OpenRouter)	Redactar las respuestas de Banu a partir de la información oficial encontrada.	Clave API en encabezado Bearer (OLLAMA_API_KEY, GROQ_API_KEY, GEMINI_API_KEY, OPENROUTER_API_KEY).	AiGatewayService: 25 s por modelo; si un modelo responde 429 descansa 2 minutos y se prueba el siguiente.
Firebase Cloud Messaging, API HTTP v1 (Google, s. f.)	Enviar la notificación push cuando un gestor publica un aviso.	Cuenta de servicio con OAuth 2.0 (FIREBASE_CREDENTIALS).	PushNotificationService: un envío por aviso al tema de la categoría; registra push_sent_at.
API REST de WordPress de unab.edu.co	Sincronizar el catálogo de programas (precios y fichas) y las noticias.	Pública; dirección en UNAB_WP_URL.	UnabWebService: lo usa el administrador en POST /api/v1/admin/programs/sync.
Google Identity (tokens de Firebase Authentication)	Iniciar sesión con la cuenta institucional @unab.edu.co.	Llaves públicas JWKS de Google; proyecto en FIREBASE_PROJECT_ID.	GoogleIdentityService: verifica la firma RS256, el emisor, la audiencia y el vencimiento del token.

Nota. Las cinco integraciones se simulan en las pruebas automatizadas, por lo que estas no dependen de internet.

Capa de Servicios

Los controladores nunca llaman a una API externa: dependen de una interfaz (por ejemplo, PushSender) cuya implementación real usa el cliente HTTP de Ktor y cuya implementación de pruebas usa MockEngine, equivalente a las fachadas falsas de Laravel. Así, una prueba de POST /api/v1/announcements verifica que se envía exactamente una notificación con el título correcto, sin contactar a Firebase.

Credenciales y Webhooks

Las credenciales se definen en variables de entorno, documentadas en el archivo .env.example, y nunca se guardan en el repositorio. Para recibir posiciones desde un celular a bordo con Traccar Client se expone el webhook POST /api/v1/webhooks/gps. Como la API no usa cookies de sesión, la protección CSRF de los formularios web no aplica (es el equivalente de la exención del middleware en Laravel); en su lugar, cada petición debe traer una firma HMAC-SHA256 calculada con el secreto GPS_WEBHOOK_SECRET y está sujeta a un límite de peticiones por minuto, siguiendo las recomendaciones de OWASP Foundation (2023).

Definición de Endpoints REST

Todos los endpoints usan el prefijo /api/v1, reciben y devuelven JSON y siguen la semántica de los métodos y códigos de estado de HTTP (Fielding et al., 2022) y el estilo arquitectónico REST (Fielding, 2000). Las rutas protegidas exigen el encabezado Authorization: Bearer con el token de acceso. Las respuestas exitosas usan la forma { "data": … } y los errores la forma { "error": mensaje, "code": código, "fields": errores por campo, "requestId": identificador }. La Tabla 21 lista los endpoints previstos.

Tabla 21

Endpoints REST Previstos

Método	URI	Acceso	Funcionalidad	Respuestas
GET	/api/v1/health	Público	Estado del servicio y de la base de datos.	200, 500
POST	/api/v1/auth/register	Público	Registra un estudiante con correo @unab.edu.co y emite su token.	201, 400, 409
POST	/api/v1/auth/login	Público	Inicia sesión con correo y contraseña y emite un token.	200, 400, 401
POST	/api/v1/auth/google	Público	Cambia un token de Google institucional por un token propio.	200, 401
POST	/api/v1/auth/logout	Autenticado	Revoca el token actual.	204, 401
GET	/api/v1/me	Autenticado	Datos del usuario, su rol y su perfil.	200, 401
PUT	/api/v1/me/profile	Estudiante	Actualiza programa, semestre, idioma y recordatorio.	200, 400, 401
GET	/api/v1/subjects	Estudiante	Lista sus materias con el horario.	200, 401
POST	/api/v1/subjects	Estudiante	Crea una materia con sus bloques; rechaza choques de horario.	201, 400, 401
GET	/api/v1/subjects/{id}	Dueño	Detalle de una materia.	200, 403, 404
PUT	/api/v1/subjects/{id}	Dueño	Modifica la materia y su horario.	200, 400, 403, 404
DELETE	/api/v1/subjects/{id}	Dueño	Elimina la materia con sus cortes y notas.	204, 403, 404
POST	/api/v1/subjects/{id}/categories	Dueño	Crea un corte; la suma de pesos no puede superar 100 %.	201, 400, 403
POST	/api/v1/categories/{id}/items	Dueño	Registra una nota entre 0,0 y 5,0.	201, 400, 403, 404
GET	/api/v1/subjects/{id}/summary	Dueño	Promedio, acumulado y nota necesaria para aprobar (Arts. 39 y 44).	200, 403, 404
GET	/api/v1/schedule/next	Estudiante	Próxima clase del estudiante.	200, 401
GET	/api/v1/programs	Público	Programas con filtros por facultad, nivel y precio máximo.	200, 400
GET	/api/v1/programs/{slug}	Público	Ficha de un programa.	200, 404
GET	/api/v1/bus/routes	Público	Rutas con sus paradas.	200
GET	/api/v1/bus/live	Público	Buses en vivo con estado, rumbo y sede más cercana (API GPS).	200, 500
PUT	/api/v1/bus/routes/{id}	Gestor	Edita una ruta y el orden de sus paradas.	200, 400, 403, 404
POST	/api/v1/me/favorite-routes/{routeId}	Autenticado	Marca una ruta como favorita.	201, 404

DELETE	/api/v1/me/favorite-routes/{routeId}	Autenticado	Quita una ruta de favoritas.	204, 404
GET	/api/v1/announcements	Público	Avisos publicados, paginados y por categoría.	200
POST	/api/v1/announcements	Gestor	Crea un aviso; si se publica, envía la notificación push (FCM).	201, 400, 403
PUT	/api/v1/announcements/{id}	Autor o administrador	Edita o publica un aviso.	200, 400, 403, 404
DELETE	/api/v1/announcements/{id}	Autor o administrador	Retira un aviso.	204, 403, 404
POST	/api/v1/banu/ask	Público con límite	Pregunta a Banu: preguntas frecuentes, búsqueda y IA verificada.	200, 400, 429
GET	/api/v1/faqs	Gestor	Lista las preguntas frecuentes.	200, 403
POST	/api/v1/faqs	Gestor	Crea una pregunta frecuente con su fuente oficial.	201, 400, 403
PUT	/api/v1/faqs/{id}	Gestor	Edita una pregunta frecuente.	200, 400, 403, 404
DELETE	/api/v1/faqs/{id}	Gestor	Desactiva una pregunta frecuente.	204, 403, 404
GET	/api/v1/admin/users	Administrador	Lista usuarios con filtros por rol.	200, 403
PATCH	/api/v1/admin/users/{id}/role	Administrador	Cambia el rol de un usuario.	200, 400, 403, 404
POST	/api/v1/admin/programs/sync	Administrador	Sincroniza programas desde unab.edu.co (WordPress).	200, 403, 500
POST	/api/v1/webhooks/gps	Firma HMAC	Recibe la posición de un dispositivo a bordo.	202, 400, 401

Nota. 400 = datos inválidos; 401 = sin token o token vencido; 403 = sin permiso; 404 = no existe; 409 = correo ya registrado; 429 = demasiadas peticiones; 500 = error interno (sin detalles técnicos en la respuesta).

Endpoints Núcleo para la Entrega Final

De la lista anterior, cinco endpoints concentran los requisitos del curso (autenticación con tokens, validación de solicitudes, reglas de negocio, autorización por rol y por propiedad, e integración con APIs externas) y serán los primeros en quedar completos y probados:

- **POST /api/v1/auth/login:** autenticación y emisión de tokens.
- **POST /api/v1/subjects:** validación de la materia y de sus bloques sin choques de horario (201 Created).

- **GET /api/v1/subjects/{id}/summary:** cálculo de notas según el Reglamento Estudiantil y política de propiedad.
- **POST /api/v1/announcements:** rol de gestor e integración con Firebase Cloud Messaging.
- **GET /api/v1/bus/live:** integración con la plataforma GPS de las Rutas UNAB, con caché.

Plan de Pruebas

Cada endpoint tendrá pruebas de integración con JUnit y el entorno testApplication de Ktor, sobre una base de datos H2 en memoria que Flyway migra antes de cada prueba. Las APIs externas se reemplazan por MockEngine. Las pruebas cubren el caso exitoso y los errores 400, 401, 403 y 404 de cada endpoint. El repositorio público del backend incluirá un archivo README.md con la instalación, la configuración de .env.example, la ejecución de migraciones y semillas y el comando de las pruebas.

Referencias

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial 48.587.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

Google. (s. f.). *Firebase Cloud Messaging*. Firebase. https://firebase.google.com/docs/cloud-messaging

JetBrains. (s. f.-a). *Ktor documentation*. https://ktor.io/docs/welcome.html

JetBrains. (s. f.-b). *Exposed documentation*. https://www.jetbrains.com/help/exposed/home.html

OWASP Foundation. (2023). *OWASP API security top 10: 2023*. https://owasp.org/API-Security/editions/2023/en/0x00-header/

Redgate. (s. f.). *Flyway documentation*. https://documentation.red-gate.com/flyway

Universidad Autónoma de Bucaramanga. (2019). *Reglamento estudiantil de pregrado*. https://unab.edu.co/wp-content/uploads/2026/04/Reglamento-estudiantil-pregrado.pdf